

Part 2: Linux SocketCAN Architecture

Automotive Networking as Native Linux Network Sockets

1. The SocketCAN Revolution

Historically, CAN drivers on Unix and Windows were implemented as character devices (`/dev/can0`) requiring proprietary DLLs, vendor-specific APIs (Vector XL, Kvaser, PEAK PCAN), and cumbersome `ioctl()` calls. A major limitation was that only one process could hold the character device open at any time.

In 2006, Volkswagen Research and the Linux community developed **SocketCAN**. SocketCAN models CAN interfaces as native Linux network devices (just like `eth0` or `wlan0`), introducing the `PF\_CAN` protocol family. Multiple independent user-space processes can concurrently read and write CAN frames through standard Berkeley socket APIs (`socket()`, `bind()`, `read()`, `write()`, `select()`).

2. Virtual CAN (`vcan`) Kernel Driver

The Linux kernel module `vcan` provides virtual loopback CAN interfaces without requiring physical hardware transceivers:

- The frame enters the Linux network subsystem through `can\_send()`.
 - The kernel replicates the frame to its loopback mechanism.
 - All other processes bound to `vcan0` receive the frame in their socket queues.
 - Kernel filters (`CAN\_RAW\_FILTER`) discard unwanted frames in kernel-space before waking user-space threads.

3. SocketCAN in C: The API Workflow

```
#include
#include
#include
#include

// 1. Create a raw CAN socket
int s = socket(PF_CAN, SOCK_RAW, CAN_RAW);

// 2. Discover interface index for "vcan0"
struct ifreq ifr;
strcpy(ifr.ifr_name, "vcan0");
ioctl(s, SIOCGIFINDEX, &ifr);

// 3. Bind socket to interface
struct sockaddr_can addr;
memset(&addr, 0, sizeof(addr));
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
bind(s, (struct sockaddr *)&addr, sizeof(addr));

// 4. Send a standard CAN frame
struct can_frame frame;
frame.can_id = 0x100;
frame.can_dlc = 8;
memcpy(frame.data, payload, 8);
write(s, &frame, sizeof(struct can_frame));
```

4. Essential can-utils Diagnostics

Command	Usage Example	Purpose
candump	candump -tz vcan0	Dumps incoming CAN traffic with timestamps.
cansend	cansend vcan1 201#0100000000000000	Transmits a single custom CAN frame manually.
cangen	cangen vcan0 -g 10	Generates random CAN traffic for load testing.
canplayer	canplayer -l trace.log	Replays recorded CAN logs accurately.